Análisis y diseño de QTZ2

1 requisitos funcionales

Código	Operación	Descripción
RF01	Carga inicial	Iniciar una misión en progreso con al menos 10 módulos de los tres tipos, recursos e historiales coherentes.
RF02	Listar módulos	Mostrar las características comunes y particulares de todos los módulos.
RF03	Buscar módulos	Buscar por ID o nombre y mostrar toda la información; informar cuando no existan coincidencias.
RF04	Consultar catálogo	Mostrar una copia ordenada por costo de construcción ascendente sin cambiar el orden de participación.
RF05	Avanzar un ciclo	Dar a cada módulo una oportunidad de operar en el orden de la carga inicial, mediante procesarCiclo.
RF06	Actualizar recursos	Aplicar cada cambio inmediatamente y conservar los resultados para el siguiente módulo y ciclo.
RF07	Mostrar resultados	Presentar la operación o causa de no operar de cada módulo y los recursos al terminar.
RF08	Resumen de comunicaciones	Mostrar cantidad de antenas, activas, capacidad activa, total histórico descargado y todas las antenas empatadas en el máximo.
RF09	Menú de consola	Permitir las cinco operaciones solicitadas y una opción adicional para salir.

2 Reglas de operación y responsabilidades

Tipo	Condiciones para operar	Cambios en un ciclo exitoso
Vuelo	Activo y energía disponible ≥ consumo.	Resta consumo completo; suma datosPorCiclo a órbita; incrementa ciclosAcumulados.
Tierra	Activo, energía suficiente y datosEnOrbita > 0.	Descarga min(capacidad, datosEnOrbita); consume toda su energía; actualiza los totales de la misión y de la antena.
Energía	Activo.	Suma energiaPorCiclo a la misión e incrementa ciclosAcumulados.

Separación MVC

Componente	Clases	Responsabilidad
Modelo	Mision, Modulo y sus tres subclases	Conservar estado, ejecutar operaciones y calcular consultas.
Vista	VistaConsola	Leer entradas y presentar información.
Controlador	Controlador	Relacionar las opciones elegidas con las operaciones del modelo y los métodos de la vista.
Inicio	Main	Crear y conectar los objetos.

Herencia y polimorfismo

ModuloEnergia, ModuloVuelo y ModuloTierra son especializaciones de Modulo. Las tres sobrescriben procesarCiclo(Mision) y toString(). Mision recorre ArrayList<Modulo> e invoca el mismo método sin decidir la operación con if según el tipo. La identificación de ModuloTierra se reserva para el informe específico de comunicaciones.

3 Clase común y módulo de vuelo

Modulo

Atributo privado	Tipo	Propósito
id	String	Identificador único e inmutable.
nombre	String	Nombre del módulo.
salud	int	Valor entre 0 y 100.
activo	boolean	Indica si puede intentar su operación.
costoConstruccion	double	Costo fijo de construcción, no negativo.

Método y visibilidad	Propósito y resultado
+ procesarCiclo(mision: Mision): String	Método abstracto. Cada subclase ejecuta su operación y devuelve el resultado o la causa de no operar.
+ getId(): String	Devuelve el ID.
+ getNombre(): String	Devuelve el nombre.
+ getSalud(): int	Devuelve la salud.

Método y visibilidad	Propósito y resultado
+ isActivo(): boolean	Devuelve el estado de actividad.
+ getCostoConstruccion(): double	Devuelve el costo.
+ toString(): String	Devuelve la información común. Las subclases la amplían con super.toString().

ModuloVuelo

Atributo privado	Tipo	Propósito
tipoInstrumento	String	Cámara o sensor.
datosPorCiclo	double	MB recolectados por operación; capacidad fija.
consumoEnergia	double	Energía necesaria para una operación completa.
ciclosAcumulados	int	Cantidad histórica de ciclos en los que operó.

Método y visibilidad	Propósito y resultado
+ procesarCiclo(mision: Mision): String	Comprueba actividad y energía; consume el costo completo, agrega los MB a órbita e incrementa sus ciclos.
+ getTipoInstrumento(): String	Devuelve el tipo de instrumento.
+ getDatosPorCiclo(): double	Devuelve la capacidad fija.
+ getConsumoEnergia(): double	Devuelve el consumo.
+ getCiclosAcumulados(): int	Devuelve los ciclos exitosos.
+ getTotalRecolectado(): double	Calcula datosPorCiclo × ciclosAcumulados, incluyendo la historia inicial.
+ toString(): String	Amplía la información común con instrumento, capacidad, consumo, ciclos y total histórico.

4 Módulos de tierra y energía

ModuloTierra

Atributo privado	Tipo	Propósito
estacion	String	Estación a la que pertenece la antena.
capacidadDescarga	double	Máximo de MB descargados por ciclo; capacidad fija.
energiaDescarga	double	Consumo completo de energía por descarga.

Atributo privado	Tipo	Propósito
totalDescargado	double	MB descargados históricamente por este módulo.

Método y visibilidad	Propósito y resultado
+ procesarCiclo(mision: Mision): String	Comprueba actividad, energía y datos; consume el costo completo, descarga el mínimo entre capacidad y MB en órbita y actualiza su historial.
+ getEstacion(): String	Devuelve la estación.
+ getCapacidadDescarga(): double	Devuelve la capacidad.
+ getEnergiaDescarga(): double	Devuelve el costo energético.
+ getTotalDescargado(): double	Devuelve los MB históricos.
+ toString(): String	Amplía la información común con estación, capacidad, consumo e historial.

ModuloEnergia

Atributo privado	Tipo	Propósito
energiaPorCiclo	double	Energía producida por operación; capacidad fija.
ciclosAcumulados	int	Cantidad histórica de ciclos en los que generó energía.

Método y visibilidad	Propósito y resultado
+ procesarCiclo(mision: Mision): String	Si está activo, agrega energía disponible e incrementa sus ciclos; si está inactivo, informa el motivo.
+ getEnergiaPorCiclo(): double	Devuelve la generación fija.
+ getCiclosAcumulados(): int	Devuelve los ciclos exitosos.
+ getTotalGenerado(): double	Calcula energiaPorCiclo × ciclosAcumulados, incluyendo la historia inicial.
+ toString(): String	Amplía la información común con generación, ciclos y total histórico.

5 Gestión de la misión

Mision

Atributo privado	Tipo	Propósito
modulos	ArrayList<Modulo>	Única colección principal, con el orden de participación.
energiaDisponible	double	Energía que pueden usar los módulos.
datosEnOrbita	double	MB pendientes de descarga.
totalDescargado	double	MB históricos descargados por toda la misión.
cicloActual	int	Cantidad de ciclos completos de la misión.

Método y visibilidad	Propósito y resultado
- cargarInicial(): void	Crea los 10 módulos e inicializa sus recursos e historiales una sola vez.
+ listarModulos(): ArrayList<Modulo>	Devuelve una copia para consulta, conservando el orden.
+ buscarPorId(id: String): Modulo	Devuelve la coincidencia por ID o null si no existe.
+ buscarPorNombre(nombre: String): ArrayList<Modulo>	Devuelve todas las coincidencias exactas sin distinguir mayúsculas.
+ obtenerCatalogoOrdenado(): ArrayList<Modulo>	Devuelve una copia ordenada por costo ascendente y por ID en los empates.
+ avanzarCiclo(): ArrayList<String>	Recorre la lista original, invoca procesarCiclo(this) e incrementa cicloActual al terminar.
+ consumirEnergia(cantidad: double): boolean	Descuenta solo si hay suficiente energía; de lo contrario devuelve false sin modificar recursos.
+ agregarEnergia(cantidad: double): void	Suma energía disponible.
+ agregarDatos(cantidad: double): void	Suma MB pendientes en órbita.
+ descargarDatos(capacidad: double): double	Retira min(capacidad, datosEnOrbita), lo suma al total de la misión y devuelve los MB retirados.
+ getCantidadTierra(): int	Cuenta todos los módulos de tierra.
+ getCantidadTierraActivos(): int	Cuenta los módulos de tierra activos.
+ getCapacidadDescargaActiva(): double	Suma las capacidades de las antenas activas.
+ obtenerMayoresDescargas(): ArrayList<ModuloTierra>	Devuelve todas las antenas cuyo historial iguala el máximo, incluidas las empatadas.
+ getEnergiaDisponible(): double	Devuelve la energía actual.
+ getDatosEnOrbita(): double	Devuelve los MB pendientes.

Método y visibilidad	Propósito y resultado
+ getTotalDescargado(): double	Devuelve el total histórico de la misión.
+ getCicloActual(): int	Devuelve el número de ciclo.

6 Controlador vista e inicialización

Controlador

Atributo privado	Tipo	Propósito
mision	Mision	Modelo que ejecuta las operaciones.
vista	VistaConsola	Vista que solicita entradas y presenta resultados.

Método y visibilidad	Propósito y resultado
+ iniciar(): void	Mantiene el menú hasta seleccionar salir y dirige cada opción al modelo y a la vista.

VistaConsola

Atributo privado	Tipo	Propósito
lector	Scanner	Lee entradas de la consola.

Método y visibilidad	Propósito y resultado
+ leerOpcion(): int	Muestra el menú y valida que la opción esté entre 0 y 5.
+ leerTipoBusqueda(): int	Solicita búsqueda por ID o por nombre.
+ leerTexto(mensaje: String): String	Solicita un texto no vacío.
+ mostrarMensaje(mensaje: String): void	Presenta mensajes y casos sin coincidencias.
+ mostrarModulos(modulos: ArrayList<Modulo>): void	Presenta los módulos con su toString polimórfico.

Método y visibilidad	Propósito y resultado
+ mostrarCiclo(resultados: ArrayList<String>, ciclo: int, energia: double, orbita: double, descargado: double): void	Presenta el resultado de cada módulo y los recursos finales.
+ mostrarResumen(total: int, activos: int, capacidad: double, descargado: double, mayores: ArrayList<ModuloTierra>): void	Presenta las métricas y el nombre, ID y estación de todas las antenas líderes.

Main

Método y visibilidad	Propósito y resultado
+ main(args: String[]): void	Crea Mision y VistaConsola, los entrega al Controlador y llama iniciar(). Método público y estático.

Colaboración de los constructores

Cada constructor de una subclase invoca super(id, nombre, salud, activo, costoConstruccion) para inicializar la parte común. Después usa this para asignar sus atributos particulares y sus acumulados iniciales. El constructor de Modulo es protegido porque se utiliza desde sus subclases y la clase es abstracta.

No se necesitan setters generales: el menú consulta información y avanza ciclos. Los recursos y contadores se modifican únicamente por las operaciones que mantienen sus reglas. El ID y las capacidades no tienen setters. Si luego se añaden activación o reparación, se incorporarán métodos específicos.

7 Carga inicial de la misión

ID	Nombre	Tipo	Salud	Activo	Costo
E01	Panel Norte	Energía	100	Sí	500
V01	Cámara UVG	Vuelo	95	Sí	1200
T01	Antena Central	Tierra	90	Sí	800
E02	Panel Sur	Energía	85	Sí	450
V02	Sensor Solar	Vuelo	90	Sí	1000
T02	Antena Oriente	Tierra	80	Sí	950

ID	Nombre	Tipo	Salud	Activo	Costo
E03	Panel Auxiliar	Energía	70	No	650
V03	Sensor Térmico	Vuelo	85	Sí	700
T03	Antena Reserva	Tierra	75	No	600
V04	Cámara Reserva	Vuelo	80	Sí	1100

Módulo	Valores particulares iniciales
E01	Genera 25 unidades por ciclo; 3 ciclos exitosos; total generado 75.
E02	Genera 20 unidades por ciclo; 2 ciclos exitosos; total generado 40.
E03	Genera 30 unidades por ciclo; 2 ciclos exitosos; total generado 60.
V01	Cámara; 15 MB por ciclo; consume 8; 2 ciclos; total recolectado 30 MB.
V02	Sensor; 20 MB por ciclo; consume 10; 3 ciclos; total recolectado 60 MB.
V03	Sensor; 10 MB por ciclo; consume 6; 4 ciclos; total recolectado 40 MB.
V04	Cámara; 20 MB por ciclo; consume 9; 2 ciclos; total recolectado 40 MB.
T01	Estación Central UVG; capacidad 10 MB; consume 5; descargado 30 MB.
T02	Estación Oriente UVG; capacidad 20 MB; consume 6; descargado 40 MB.
T03	Estación Reserva UVG; capacidad 20 MB; consume 7; descargado 20 MB.

8 Ejecución de ciclos e informe

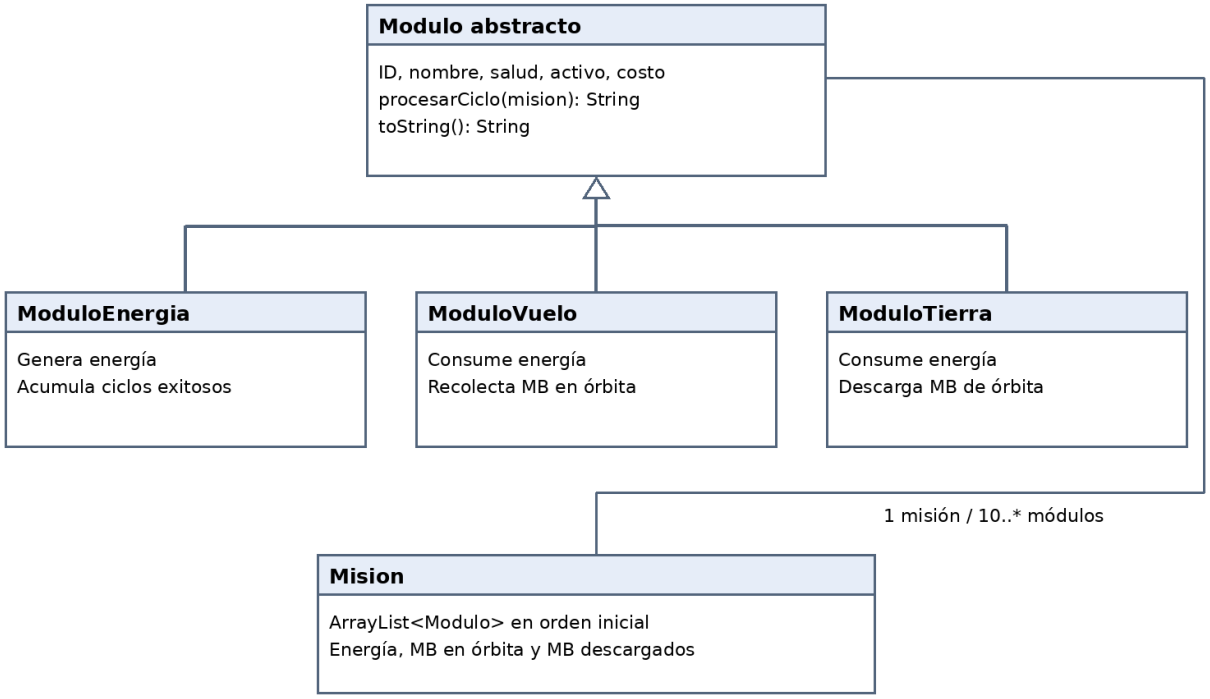
Orden	Módulo y resultado	Energía	Órbita MB	Descargado MB
Inicio	Recursos antes del ciclo	53	80	90
1	E01 genera 25	78	80	90
2	V01 consume 8 y recolecta 15	70	95	90
3	T01 consume 5 y descarga 10	65	85	100
4	E02 genera 20	85	85	100
5	V02 consume 10 y recolecta 20	75	105	100
6	T02 consume 6 y descarga 20	69	85	120
7	E03 está inactivo	69	85	120
8	V03 consume 6 y recolecta 10	63	95	120

Orden	Módulo y resultado	Energía	Órbita MB	Descargado MB
9	T03 está inactivo	63	95	120
10	V04 consume 9 y recolecta 20	54	115	120

Resumen de comunicaciones

Dato	Carga inicial	Después del ciclo 6
Cantidad de módulos de tierra	3	3
Módulos de tierra activos	2	2
Capacidad total activa	30 MB por ciclo	30 MB por ciclo
Total histórico descargado	90 MB	120 MB
Mayor historial	T02 Antena Oriente 40 MB	T02 Antena Oriente 60 MB
Estación del módulo líder	Estación Oriente UVG	Estación Oriente UVG

9 Relaciones y vista general del UML



Relación	Significado
Herencia hacia Modulo	Cada subclase comparte datos y sobrescribe su comportamiento.
Mision con Modulo	Una misión mantiene al menos 10 módulos ordenados en su colección principal.
Modulo usa Mision	procesarCiclo recibe los recursos compartidos como parámetro.
Controlador con Mision y VistaConsola	Conserva una referencia al modelo y una a la vista.
Main usa las clases MVC	Crea sus objetos y arranca la aplicación.

10 Casos de verificación previstos

Caso	Resultado esperado
Carga inicial	Hay 10 módulos de los tres tipos en una colección polimórfica y sus historiales coinciden con los recursos.
Búsqueda por ID inexistente	Se presenta un mensaje sin alterar la misión.
Nombres repetidos	Se muestran todas las coincidencias.
Catálogo ordenado	Se muestran costos ascendentes y se mantiene el orden de ejecución original.
Vuelo con energía insuficiente	No cambia la energía, los MB en órbita ni los ciclos acumulados.
Antena con 3 MB y capacidad de 10 MB	Descarga 3 MB y consume el costo energético completo.
Antena sin datos en órbita	No consume energía ni cambia su historial.
Módulo inactivo	Informa la causa y no modifica recursos ni contadores.
Empate entre antenas	El resumen muestra nombre, ID y estación de todas las que comparten el máximo.
Primer ciclo desde la carga propuesta	El ciclo 6 termina con 54 unidades de energía, 115 MB en órbita y 120 MB históricos descargados.
Segundo ciclo consecutivo	Comienza con los valores del anterior y termina en el ciclo 7 con 55 unidades, 150 MB en órbita y 150 MB descargados.
Entradas inválidas en consola	La vista vuelve a solicitar la entrada sin interrumpir el menú.